

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO HỌC PHẦN
DỰ ÁN

Đề tài

ĐIỀU PHỐI XUNG ĐỘT BỘ NHỚ
CHO HỆ ĐA TÁC TỬ

Giảng viên hướng dẫn: Đào Việt Anh

Họ và tên: Ngô Quang Dũng
Mã sinh viên: 23020344
Lớp: K68-AI2

Hà Nội, 2026

Ý kiến đánh giá:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm số

Điểm chữ

Hà Nội, ngày ..., tháng ..., năm 20....

Giảng viên đánh giá
(Ký, ghi rõ họ tên)

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành tới giảng viên hướng dẫn Đào Việt Anh đã định hướng và góp ý trong quá trình hình thành đề tài, xây dựng kiến trúc hệ thống và xác định phạm vi thực nghiệm phù hợp với quy mô học phần. Những góp ý này giúp em tập trung vào phần cốt lõi của dự án: cơ chế điều phối bộ nhớ và giải quyết xung đột trong hệ đa tác tử, thay vì chỉ tối ưu theo một benchmark cụ thể.

Em cũng xin cảm ơn cộng đồng nghiên cứu và mã nguồn mở đã cung cấp nhiều ý tưởng nền tảng về hệ thống đa tác tử, bộ nhớ dùng chung, retrieval-augmented generation, kiểm thử phần mềm và đánh giá thực nghiệm.

Trong quá trình thực hiện, báo cáo chắc chắn vẫn còn những hạn chế do phạm vi dự án ở mức học phần và kết quả thực nghiệm chưa được mở rộng trên nhiều benchmark thực tế. Em kính mong nhận được nhận xét và góp ý của thầy để tiếp tục hoàn thiện hướng nghiên cứu này.

Em xin chân thành cảm ơn!

TỔNG QUAN

Trong thời kỳ mà hệ đa tác tử (Multi-agent systems) được đẩy mạnh [1, 2]. Việc xảy ra mâu thuẫn trong lúc các đa tác tử làm việc với nhau thường xuyên xảy ra. Trong các nghiên cứu về đề tài này, đa phần đều phụ thuộc vào LLM end-to-end, phương pháp đạt được kết quả khá tốt, nhưng mà chi phí tiêu tốn cũng vô cùng cao. Báo cáo này trình bày dự án *Memory Conflict Arbitration System*, một hệ thống bộ nhớ dùng chung cho hai tác tử độc lập. Trong hệ thống, mỗi tác tử đọc một phần khác nhau của hội thoại, trích xuất các belief hoặc fact, sau đó ghi vào *shared memory*. Khi belief mới mâu thuẫn với belief đã tồn tại, hệ thống không ghi đè đơn giản theo kiểu “last write wins”, mà thực hiện một chuỗi xử lý gồm phát hiện xung đột, phân loại xung đột theo taxonomy C1–C4, định tuyến sang policy phù hợp bằng *AdaptivePolicy*, cập nhật working memory và lưu lại audit trail.

Bằng cách đó, những trí nhớ sẽ được xử lý chính xác mà không cần LLM can thiệp quá nhiều, từ đó tiết kiệm lượng lớn chi phí chi trả cho quá trình này. Về mặt kiến trúc, dự án tách rõ *Mechanism Layer* và *Adapter Layer*. *Mechanism Layer* chứa các thành phần tổng quát như *BeliefVersion*, *MemorySlot*, *SharedMemory*, *ConflictClassifier*, các policy P1–P5 và *AdaptivePolicy*. *Adapter Layer* chịu trách nhiệm biến cơ chế tổng quát thành pipeline cho benchmark đánh giá EM/F1 (Những metric chính của dự án).

MỤC LỤC

LỜI CẢM ƠN	3
TỔNG QUAN	4
DANH MỤC VIẾT TẮT VÀ THUẬT NGỮ	8
1. GIỚI THIỆU	10
1.1. Đặt vấn đề	10
1.2. Mục tiêu nghiên cứu	10
1.3. Câu hỏi nghiên cứu	11
2. CÁC NGHIÊN CỨU LIÊN QUAN	11
2.1. Tác tử LLM và xu hướng xử lý end-to-end	11
2.2. Bộ nhớ dài hạn và truy hồi thông tin trong hệ LLM	12
3. CƠ SỞ LÝ THUYẾT VÀ MÔ HÌNH VẤN ĐỀ	12
3.1. Bộ nhớ dùng chung trong hệ đa tác tử	12
3.2. Mô hình formal của belief	13
3.3. Theta và thứ tự thời gian logic	14
3.4. Taxonomy xung đột C1–C4	14
4. KIẾN TRÚC HỆ THỐNG	15
4.1. Nguyên tắc tách Mechanism Layer và Adapter Layer	15
4.2. SharedMemory và MemorySlot	16
4.3. Hai agent độc lập	17
4.4. Interaction Protocol	17
4.5. Temporal Split Module	18
5. CƠ CHẾ PHÁT HIỆN VÀ GIẢI QUYẾT XUNG ĐỘT	19
5.1. ConflictClassifier	19
5.2. Các chính sách phân xử P1–P5	19
5.3. AdaptivePolicy	20
6. THIẾT KẾ THỰC NGHIỆM VÀ TRIỂN KHAI	21
6.1. Dữ liệu đánh giá và thiết lập thực nghiệm	21
6.2. Baseline và chỉ số đánh giá	21
6.3. Kết quả thực nghiệm	22
6.4. Đối chiếu câu hỏi nghiên cứu với kết quả đạt được	22
7. KẾT LUẬN	24
TÀI LIỆU THAM KHẢO	25

DANH SÁCH HÌNH ẢNH

Hình 1:	Phân tách giữa lớp cơ chế lỗi và lớp thích nghi dữ liệu	16
Hình 2:	Quy trình xử lý của hệ thống Memory Conflict Arbitration	18

DANH SÁCH BẢNG

Bảng 2:	Ý nghĩa các trường chính trong BeliefVersion.	14
Bảng 3:	Taxonomy C1–C4 của conflict trong shared memory.	15
Bảng 4:	Các chính sách phân xử và phạm vi áp dụng.	20
Bảng 5:	Kết quả trên MemoryAgentBench CR split với gpt-4o-mini.	22
Bảng 6:	Mức cải thiện độ chính xác và chi phí token.	22
Bảng 7:	Mức độ giải quyết các câu hỏi nghiên cứu trong phạm vi dự án.	23

DANH MỤC VIẾT TẮT VÀ THUẬT NGỮ

Chữ viết tắt / Thuật ngữ	Nguyên nghĩa
LLM	Large Language Model (Mô hình ngôn ngữ lớn).
RAG	Retrieval-Augmented Generation (Sinh văn bản có tăng cường truy hồi thông tin).
Agent	Tác tử xử lý một phần thông tin, trích xuất belief và ghi vào bộ nhớ dùng chung.
SharedMemory	Bộ nhớ dùng chung giữa các tác tử, chịu trách nhiệm lưu belief, phát hiện xung đột và cập nhật trạng thái sau phân xử.
BeliefVersion	Phiên bản thông tin được tác tử ghi vào bộ nhớ, gồm key, value, source, confidence, θ , evidence và conflict type.
MemorySlot	Đơn vị lưu trữ theo từng key trong bộ nhớ, gồm working value, danh sách phiên bản belief và trạng thái phân xử.
θ	Logical temporal order (thứ tự thời gian logic), dùng để xác định quan hệ trước-sau giữa các belief.
C1	Temporal obsolescence (xung đột do thông tin cũ lỗi thời): fact cũ từng đúng nhưng đã bị thay thế bởi fact mới hơn.
C2	Source disagreement (bất đồng nguồn): hai nguồn gần cùng thời điểm đưa ra hai giá trị khác nhau cho cùng một key.
C3	Scope mismatch (lệch phạm vi): hai thông tin tưởng như mâu thuẫn nhưng thực ra đúng trong hai phạm vi hoặc ngữ cảnh khác nhau.
C4	Entity ambiguity (nhập nhằng thực thể): cùng một key hoặc tên bề mặt nhưng thực chất trỏ tới hai thực thể khác nhau.
P1	OverwritePolicy (chính sách ghi đè): thay thế giá trị cũ bằng belief mới khi có tín hiệu cập nhật rõ ràng.
P2	ParallelPolicy (chính sách giữ song song): giữ đồng thời nhiều giá trị khi chưa đủ cơ sở chọn một giá trị duy nhất.
P3	QuarantinePolicy (chính sách cách ly): tạm giữ belief chưa đủ tin cậy và chưa đưa vào working memory.

Chữ viết tắt / Thuật ngữ	Nguyên nghĩa
P4	SoftMergePolicy (chính sách hợp nhất mềm): kết hợp các giá trị khi chúng có thể được hợp nhất, nhất là với dữ liệu định lượng.
P5	ContextualizePolicy (chính sách ngữ cảnh hóa): lưu giá trị theo scope hoặc context riêng thay vì loại bỏ một trong hai.
AdaptivePolicy	Bộ định tuyến chính sách, lựa chọn P1–P5 dựa trên loại xung đột C1–C4.
EM	Exact Match (khớp chính xác), chỉ số đánh giá câu trả lời bằng cách so khớp chính xác với đáp án chuẩn.
F1	F1-score, trung bình điều hòa giữa precision và recall, thường dùng khi đánh giá mức độ trùng khớp token hoặc phân loại.
CR-Acc	Conflict Resolution Accuracy (độ chính xác giải quyết xung đột), phản ánh tỷ lệ câu trả lời đúng sau khi hệ thống xử lý conflict.
Token cost	Chi phí token, thường được tính bằng số token trung bình sử dụng khi đưa ngữ cảnh vào LLM để trả lời truy vấn.
End-to-end	Cách xử lý trong đó LLM nhận trực tiếp ngữ cảnh hoặc lịch sử truy hồi và tự suy luận câu trả lời, thay vì có một lớp phân xử bộ nhớ riêng biệt.

1 GIỚI THIỆU

1.1 Đặt vấn đề

Các hệ thống đa tác tử ngày càng được sử dụng trong những bài toán cần chia nhỏ công việc, phân vai xử lý thông tin và tổng hợp kết quả từ nhiều nguồn [1, 2]. Trong các hệ thống này, mỗi tác tử thường chỉ quan sát được một phần ngữ cảnh hoặc một giai đoạn của hội thoại. Khi các tác tử cùng ghi thông tin vào một bộ nhớ chung, vấn đề không chỉ nằm ở việc lưu trữ fact, mà còn nằm ở cách xử lý khi các fact đó không tương thích với nhau.

Một ví dụ đơn giản là trường hợp Agent A đọc phần đầu hội thoại và ghi belief rằng địa chỉ của người dùng là địa chỉ cũ. Sau đó, Agent B đọc phần sau hội thoại và phát hiện người dùng đã chuyển sang địa chỉ mới. Nếu hệ thống chỉ giữ fact cũ, câu trả lời cuối cùng sẽ lỗi thời. Nếu hệ thống luôn ghi đè bằng fact mới, nó có thể xử lý đúng trường hợp temporal update, nhưng lại sai trong trường hợp hai nguồn bất đồng gần như đồng thời hoặc hai fact cùng đúng trong hai scope khác nhau. Vì vậy, bài toán không thể được giải quyết bằng một quy tắc đơn giản duy nhất.

Trong thực tế, conflict trong memory có nhiều nguyên nhân khác nhau. Có trường hợp world state thay đổi theo thời gian, có trường hợp hai nguồn bất đồng, có trường hợp hai fact cùng đúng nhưng thuộc hai scope khác nhau, và có trường hợp một key đang gộp nhầm hai thực thể khác nhau. Nếu không phân biệt các nguyên nhân này, hệ thống dễ đưa ra hành động sai: overwrite khi cần contextualize, parallel khi cần overwrite, hoặc merge khi cần split key.

Dự án *Memory Conflict Arbitration System* được xây dựng để xử lý chính vấn đề đó. Hệ thống mô phỏng một shared memory cho hai agent độc lập. Agent A xử lý phần đầu hội thoại, Agent B xử lý phần sau. Mỗi agent chỉ trích xuất belief với metadata và không được tự gán conflict type. Khi belief mới được ghi vào memory, kiểm tra conflict dựa trên cặp belief (*existing, incoming*), gọi classifier để phân loại C1–C4, sau đó dùng AdaptivePolicy để chọn policy phù hợp. Kết quả cuối cùng được commit vào working memory và được lưu audit trail để không mất thông tin cũ.

1.2 Mục tiêu nghiên cứu

Mục tiêu tổng quát của dự án là xây dựng và đánh giá một cơ chế điều phối xung đột bộ nhớ cho hệ đa tác tử ở quy mô học phần, có khung kiến trúc rõ ràng, không phụ thuộc vào benchmark cụ thể và có thể kiểm chứng bằng thực nghiệm. Dự án không đặt mục tiêu trở thành một hệ thống production hoàn chỉnh hoặc một paper-ready system ngay từ đầu. Thay vào đó, trọng tâm là chứng minh cơ chế memory control và conflict routing có hướng đi hợp lý.

Cụ thể, dự án có bốn mục tiêu chính. Thứ nhất, định nghĩa một mô hình belief đủ rõ ràng để mọi fact ghi vào memory đều có key, value, confidence, source, θ , evidence và conflict type. Trong đó, agents không được tự phân loại conflict; conflict type chỉ được sinh ra bởi classifier sau khi memory phát hiện có mâu thuẫn.

Thứ hai, xây dựng taxonomy C1–C4 để phân biệt bốn nguyên nhân gốc của xung đột: temporal obsolescence (Thông tin cũ từng đúng, nhưng đã bị thay thế bởi thông tin mới), source disagreement (Hai nguồn gần cùng thời điểm nói khác nhau, chưa biết ai đúng), scope mismatch (Tưởng là mâu thuẫn, nhưng thật ra hai thông tin đúng trong hai ngữ cảnh khác nhau) và entity ambiguity (Cùng một key/tên nhưng thực ra nói về hai thực thể khác nhau). Taxonomy này phải nằm trong Mechanism Layer, mang tính tổng quát và không được tuning bằng keyword của một benchmark cụ thể.

Thứ ba, thiết kế các arbitration policies P1–P5 và một AdaptivePolicy đóng vai trò meta-policy. AdaptivePolicy không phải là một policy mới thay thế toàn bộ các policy khác, mà là bộ định tuyến: C1 route sang overwrite, C2 route sang parallel hoặc credibility-based overwrite, C3 route sang contextualize, C4 route sang split key, và trường hợp không chắc chắn route sang quarantine.

Thứ tư, thiết kế evaluation để kiểm tra claim trung tâm. Kết quả quan trọng không chỉ là tổng EM/F1, mà là breakdown theo từng conflict type. Nếu evaluation set không có đủ C2/C3/C4, claim “AdaptivePolicy tốt hơn vì routing” sẽ chưa được kiểm tra đúng. Vì vậy, báo cáo nhấn mạnh yêu cầu tạo natural subset và injected subset có kiểm soát, đồng thời phải document rõ tỷ lệ natural và injected.

1.3 Câu hỏi nghiên cứu

Từ các mục tiêu trên, dự án tập trung vào bốn câu hỏi nghiên cứu.

RQ1: *Liệu một cơ chế shared memory có thể phát hiện conflict dựa trên cặp belief thay vì dựa trên từng belief đơn lẻ hay không?*

RQ2: *Liệu taxonomy C1–C4 có đủ để biểu diễn các root causes chính của conflict trong hệ hai agent hay không?* RQ3: *Liệu AdaptivePolicy có vượt các single-policy baseline khi kết quả được phân tích theo từng conflict type hay không?*

RQ4: *Kiến trúc extract–arbitrate–query có thể giảm token cost so với baseline end-to-end cùng backbone model trong hội thoại dài hay không?*

2 CÁC NGHIÊN CỨU LIÊN QUAN

2.1 Tác tử LLM và xu hướng xử lý end-to-end

Các nghiên cứu gần đây về tác tử LLM cho thấy mô hình ngôn ngữ lớn có thể được tổ chức thành các hệ thống biết quan sát, lập luận, hành động và tự điều chỉnh qua nhiều bước. ReAct đề xuất cách để LLM sinh xen kẽ giữa lập luận và hành động, nhờ đó mô hình có thể vừa suy nghĩ về kế hoạch, vừa gọi công cụ hoặc truy vấn môi trường để bổ sung thông tin [2]. Reflexion tiếp tục mở rộng hướng này bằng cách cho tác tử lưu lại phản hồi dưới dạng ngôn ngữ trong bộ nhớ kinh nghiệm, rồi dùng các phản hồi đó để cải thiện quyết định ở những lượt sau [3].

Trong hệ đa tác tử, CAMEL đề xuất mô hình role-playing giữa các tác tử giao tiếp

bằng ngôn ngữ tự nhiên để nghiên cứu khả năng hợp tác và hành vi xã hội của các agent [4]. AutoGen cũng đi theo hướng xây dựng ứng dụng LLM thông qua nhiều tác tử có thể trò chuyện, gọi công cụ, viết mã và phối hợp để hoàn thành nhiệm vụ [5]. Điểm chung của các hướng này là phần lớn quá trình phối hợp vẫn dựa nhiều vào năng lực suy luận end-to-end của LLM: mô hình đọc hội thoại, sinh phản hồi, tự điều chỉnh kế hoạch và thường tự quyết định thông tin nào nên được dùng.

Cách tiếp cận end-to-end có ưu điểm là linh hoạt, dễ mở rộng sang nhiều nhiệm vụ và tận dụng trực tiếp năng lực của mô hình nền. Tuy nhiên, với bài toán bộ nhớ dùng chung, cách tiếp cận này có một hạn chế quan trọng: khi nhiều tác tử cùng ghi thông tin vào memory, xung đột giữa các belief thường không được tách thành một bước xử lý rõ ràng. Khi đó, hệ thống có thể phải đưa toàn bộ lịch sử hoặc nhiều đoạn truy hồi vào LLM để mô hình tự suy luận, dẫn đến chi phí token lớn, khó kiểm soát quyết định ghi đè và khó truy vết vì sao một thông tin được chọn thay cho thông tin khác. Đây là khoảng trống mà dự án này muốn xử lý bằng một cơ chế phân xử bộ nhớ riêng, đặt giữa bước trích xuất belief và bước trả lời truy vấn.

2.2 Bộ nhớ dài hạn và truy hồi thông tin trong hệ LLM

Retrieval-Augmented Generation (RAG) cho thấy việc kết hợp bộ nhớ tham số của mô hình với bộ nhớ ngoài có thể cải thiện các nhiệm vụ cần tri thức, đặc biệt khi thông tin cần dùng không nằm sẵn hoặc không được cập nhật trong tham số của mô hình [6]. Theo hướng tương tự, MemGPT xem LLM như một hệ điều hành có thể quản lý nhiều tầng bộ nhớ, từ đó hỗ trợ hội thoại dài và phân tích tài liệu vượt quá giới hạn context window [7].

Generative Agents cũng nhấn mạnh vai trò của memory trong hành vi tác tử. Hệ thống này lưu lại trải nghiệm của agent, truy hồi ký ức liên quan, tạo reflection và dùng chúng cho lập kế hoạch hành vi [1]. Các nghiên cứu này đều cho thấy memory là thành phần quan trọng trong tác tử LLM. Tuy nhiên, trọng tâm chính của chúng thường là lưu trữ, truy hồi, tóm tắt hoặc phản ánh lại thông tin; phần xử lý khi hai phiên bản thông tin mâu thuẫn cùng tồn tại chưa được tách thành một cơ chế phân loại và phân xử độc lập.

Vì vậy, dự án này không phủ nhận vai trò của RAG hay memory-augmented LLM. Ngược lại, hệ thống vẫn có thể sử dụng LLM ở bước trích xuất và trả lời. Điểm khác biệt nằm ở chỗ phần quyết định “ghi đè, giữ song song, cách ly, hợp nhất hay tách ngữ cảnh” được đưa về một lớp *memory arbitration* có cấu trúc, thay vì để LLM tự xử lý toàn bộ trong một prompt dài.

3 CƠ SỞ LÝ THUYẾT VÀ MÔ HÌNH VẤN ĐỀ

3.1 Bộ nhớ dùng chung trong hệ đa tác tử

Trong hệ đa tác tử, shared memory là nơi các agent ghi và đọc thông tin chung, có liên hệ với hướng tiếp cận bộ nhớ ngoài trong các hệ thống LLM [6, 7]. Nếu mỗi agent có

memory riêng, hệ thống có thể tránh được conflict ở mức lưu trữ, nhưng lại gặp khó khăn khi cần tổng hợp câu trả lời cuối cùng. Ngược lại, nếu mọi agent cùng ghi vào một memory chung mà không có arbitration, hệ thống dễ rơi vào tình trạng fact mới ghi đè fact cũ theo thứ tự vật lý, bất kể bản chất xung đột là gì.

Dự án chọn thiết kế hai agent độc lập với shared memory có điều phối. Agent A đóng vai trò temporal predecessor, xử lý phần đầu hội thoại. Agent B đóng vai trò temporal successor, xử lý phần sau hội thoại. Cả hai agent đều chỉ trích xuất belief và metadata. Việc phát hiện conflict, phân loại conflict và giải quyết conflict thuộc về memory mechanism, không thuộc về agent. Ràng buộc này giúp tránh information leakage và tránh để agent tự quyết định loại conflict dựa trên prompt tùy tiện.

Về mặt phương pháp luận, thiết kế này tạo ra một distributed simulation có chủ ý. Agent B không đọc memory của Agent A trong lúc processing, nên proposed method thực chất bị đặt vào tình huống khó hơn baseline đọc full context. Nếu hệ thống vẫn giữ được accuracy gap chấp nhận được so với baseline end-to-end, claim về lợi ích của arbitration càng có giá trị.

3.2 Mô hình formal của belief

Đơn vị ghi vào memory là BeliefVersion. Một belief gồm key, value, source, confidence, θ , evidence và conflict type. Trong đó key biểu diễn thuộc tính của thực thể, ví dụ user:address; value là giá trị belief; source là agent ghi belief; confidence nằm trong khoảng $[0, 1]$; θ là logical temporal order; evidence là câu hoặc đoạn gốc làm bằng chứng. Trường conflict_type luôn là None khi agent ghi vào memory.

Conflict xảy ra khi belief đang có và belief mới có cùng key nhưng khác value. Tuy nhiên, việc phát hiện key-value khác nhau mới chỉ là bước đầu. Sau đó, classifier phải nhìn cả cặp (*existing*, *incoming*), cùng các signal như theta gap, source identity, state-change signal, scope compatibility và entity resolution score để gán C1–C4. Vì vậy, conflict type không được suy ra từ keyword của một belief đơn lẻ.

Bảng 2: Ý nghĩa các trường chính trong BeliefVersion.

Trường	Ý nghĩa
key	Thuộc tính thực thể, ví dụ <code>user:address</code> , <code>person:job</code> , <code>company:location</code> .
value	Giá trị belief được trích xuất từ hội thoại hoặc nguồn dữ liệu.
confidence	Độ tin cậy của belief trong khoảng từ 0.0 đến 1.0; chỉ nên dùng như tiebreaker, không phải tín hiệu quyết định duy nhất.
source	Agent hoặc nguồn đã ghi belief, ví dụ <code>agent_A</code> hoặc <code>agent_B</code> .
theta	Logical temporal order; trong MemoryAgentBench có thể map thành <code>turn_index</code> .
evidence	Câu hoặc đoạn văn gốc làm bằng chứng cho belief.
conflict_type	Luôn là None khi agent ghi; chỉ được classifier gán sau khi so sánh cặp belief.

3.3 Theta và thứ tự thời gian logic

Trong dự án, θ không phải wall-clock timestamp. Nó là một logical clock biểu diễn thứ tự thời gian trừu tượng của belief [8]. Cơ chế chỉ yêu cầu θ có total order, tính được khoảng cách $\Delta\theta = \theta_{incoming} - \theta_{existing}$, và cùng đơn vị với threshold τ . Adapter Layer chịu trách nhiệm map θ sang đơn vị cụ thể. Với MemoryAgentBench, lựa chọn hợp lý là $\theta := \text{turn_index}$.

Việc dùng logical clock giúp hệ thống không phụ thuộc vào thứ tự ghi vật lý vào dict hoặc memory. Nếu một belief có θ lớn hơn và có state-change signal rõ ràng, nó có thể là update hợp lệ ngay cả khi thứ tự insert trong bộ nhớ bị đảo. Đây là điểm quan trọng để thỏa mãn property *source independence*: kết quả cuối cùng không nên phụ thuộc vào việc agent nào ghi trước ở mức implementation, mà phải phụ thuộc vào metadata có ý nghĩa như thời gian logic, confidence và source.

3.4 Taxonomy xung đột C1–C4

Hệ thống phân conflict thành bốn root causes. Taxonomy này được xây dựng từ lý thuyết xử lý xung đột bộ nhớ và belief revision, không phải từ việc quan sát rồi tuning theo một benchmark cụ thể [9, 10].

Bảng 3: Taxonomy C1–C4 của conflict trong shared memory.

Loại	Tên	Bản chất	Cách xử lý chính
C1	Temporal obsolescence	Fact cũ đúng ở thời điểm cũ, nhưng world state đã thay đổi ở thời điểm mới.	Overwrite theo update mới nếu signal đủ rõ.
C2	Source disagreement	Hai nguồn gần như đồng thời đưa ra hai giá trị khác nhau cho cùng key.	Credibility/corroboation hoặc giữ parallel.
C3	Scope mismatch	Hai fact có vẻ mâu thuẫn nhưng thực ra đúng ở hai scope hoặc context khác nhau.	Contextualize và lưu theo scope.
C4	Entity ambiguity	Một key đang trỏ tới hai thực thể khác nhau do nhập nhầm tên hoặc entity.	Split key hoặc entity disambiguation.

C1 là trường hợp phổ biến trong hội thoại dài. Người dùng có thể thay đổi địa chỉ, công việc, sở thích hoặc lịch trình. Fact cũ không nhất thiết sai; nó chỉ lỗi thời. Signal lý thuyết của C1 là $\Delta\theta > \tau$ và incoming evidence có state-change signal. Tuy nhiên, state-change keyword chỉ được nằm ở Adapter Layer, không được hardcode trong Mechanism Layer.

C2 là trường hợp hai nguồn bất đồng nhưng không có bằng chứng rõ ràng một fact mới hơn fact kia. Với C2, overwrite đơn giản có thể nguy hiểm. Nếu confidence gap lớn hoặc có corroboration độc lập, hệ thống có thể chọn winner. Nếu hai nguồn gần ngang nhau, ParallelPolicy phù hợp hơn vì nó giữ cả hai giá trị và để read strategy chọn theo confidence hoặc yêu cầu thêm bằng chứng.

C3 và C4 là pseudo-conflict. Với C3, hai fact cùng đúng nhưng thuộc hai scope khác nhau, ví dụ địa chỉ nhà và địa chỉ công ty. Với C4, hai fact có cùng surface key nhưng thực ra nói về hai entity khác nhau, ví dụ hai người cùng tên. Hai loại này cho thấy vì sao single overwrite policy không đủ: overwrite sẽ làm mất một fact đúng.

4 KIẾN TRÚC HỆ THỐNG

4.1 Nguyên tắc tách Mechanism Layer và Adapter Layer

Kiến trúc của hệ thống được thiết kế theo hướng phân tách rõ giữa lớp cơ chế lõi (*Mechanism Layer*) và lớp thích nghi dữ liệu (*Adapter Layer*). Mục đích của cách tổ chức này là giữ cho phần xử lý xung đột bộ nhớ có tính tổng quát, không phụ thuộc trực tiếp vào cấu trúc của một bộ dữ liệu hay một benchmark cụ thể.

Mechanism Layer đảm nhiệm các thành phần cốt lõi của hệ thống, bao gồm mô hình biểu diễn belief, cấu trúc MemorySlot, bộ nhớ dùng chung SharedMemory, giao diện phân loại xung đột ConflictClassifier, các chính sách phân xử P1–P5 và bộ định tuyến

AdaptivePolicy. Lớp này chỉ định nghĩa logic xử lý ở mức khái quát, do đó không chứa các luật dựa trên từ khóa hay các thao tác phụ thuộc riêng vào MemoryAgentBench.

Ngược lại, *Adapter Layer* chịu trách nhiệm ánh xạ dữ liệu thực nghiệm vào các cấu trúc mà Mechanism Layer yêu cầu. Trong trường hợp MemoryAgentBench, lớp này thực hiện các thao tác như tải mẫu dữ liệu, ánh xạ chỉ số thời gian logic θ sang turn_index, chia hội thoại theo tín hiệu cập nhật thời gian, gọi mô hình gpt-4o-mini cho bước trích xuất hoặc truy vấn, đồng thời tính các chỉ số chính gồm CR-Acc, Exact Match (EM), số token trung bình trên mỗi câu hỏi và tỷ lệ chi phí so với baseline cùng mô hình. Trong triển khai của báo cáo, kết quả được quy về CR-Acc/EM theo evaluator nội bộ, trong đó câu trả lời được so khớp với đáp án chuẩn ở mức exact match. Tuy nhiên, khi đối chiếu với protocol gốc của MemoryAgentBench, cần phân biệt rõ giữa exact match và substring exact match để tránh nhầm lẫn với thiết lập đánh giá chuẩn. Bên cạnh đó, token cost được dùng để đánh giá liệu việc hợp nhất bộ nhớ trước khi truy vấn có giúp giảm độ dài ngữ cảnh đưa vào LLM hay không.

Mechanism Layer	Adapter Layer
BeliefVersion, MemorySlot	Dataset loader, temporal split
SharedMemory	Map θ sang turn index
ConflictClassifier interface	State-change keyword detector
P1–P5 policies	Agent extraction/query
AdaptivePolicy	EM/F1/token evaluation

Hình 1: Phân tách giữa lớp cơ chế lõi và lớp thích nghi dữ liệu

4.2 SharedMemory và MemorySlot

SharedMemory là thành phần quản lý trạng thái bộ nhớ dùng chung của hệ thống. Mỗi khóa thông tin trong bộ nhớ được biểu diễn bởi một MemorySlot. Một slot bao gồm giá trị đang được sử dụng để trả lời truy vấn (working_value), danh sách các phiên bản belief đã từng được ghi (versions), trạng thái hiện tại của slot và hành động xử lý gần nhất.

Cách biểu diễn này cho phép hệ thống tách biệt giữa trạng thái bộ nhớ đang được sử dụng và lịch sử các belief đã được ghi nhận. Ngay cả khi một belief cũ bị thay thế bởi belief mới trong working_value, phiên bản cũ vẫn được lưu trong versions. Điều này giúp duy trì dấu vết xử lý, hỗ trợ phân tích lỗi và tránh việc mất hoàn toàn thông tin sau quá trình ghi đè.

Khi một belief mới được ghi vào bộ nhớ thông qua SharedMemory.write(incoming), hệ thống xử lý theo từng trường hợp. Nếu khóa chưa tồn tại, một slot mới được tạo và belief được commit trực tiếp. Nếu khóa đã tồn tại nhưng giá trị không thay đổi, belief mới được xem là một phiên bản củng cố cho thông tin hiện có. Trong trường hợp khóa trùng nhưng giá trị khác nhau, hệ thống xác định đây là một xung đột và gọi ConflictClassifier để phân loại xung đột vào một trong bốn nhóm

C1–C4. Sau đó, chính sách phân xử tương ứng được áp dụng để cập nhật `working_value`, trạng thái slot và conflict log.

Thao tác đọc từ bộ nhớ thông qua `SharedMemory.read(key)` phụ thuộc vào trạng thái của slot. Với trạng thái `quarantined`, hệ thống ưu tiên trả về giá trị cũ nhằm tránh sử dụng một belief chưa đủ tin cậy. Với trạng thái `parallel`, bộ nhớ có thể giữ đồng thời nhiều giá trị và lựa chọn phiên bản phù hợp theo độ tin cậy hoặc chuyển tiếp danh sách giá trị cho module truy vấn. Với các trạng thái `committed` và `contextual`, hệ thống trả về `working_value` đã được phân xử.

4.3 Hai agent độc lập

Hệ thống sử dụng hai tác tử độc lập, ký hiệu là Agent A và Agent B. Agent A đóng vai trò *Temporal Predecessor Agent*, xử lý phần hội thoại xuất hiện trước thời điểm cập nhật. Agent B đóng vai trò *Temporal Successor Agent*, xử lý phần hội thoại xuất hiện sau thời điểm cập nhật. Thiết kế này mô phỏng bối cảnh trong đó các tác tử chỉ quan sát được những phần ngữ cảnh khác nhau, nhưng cùng ghi thông tin vào một bộ nhớ dùng chung.

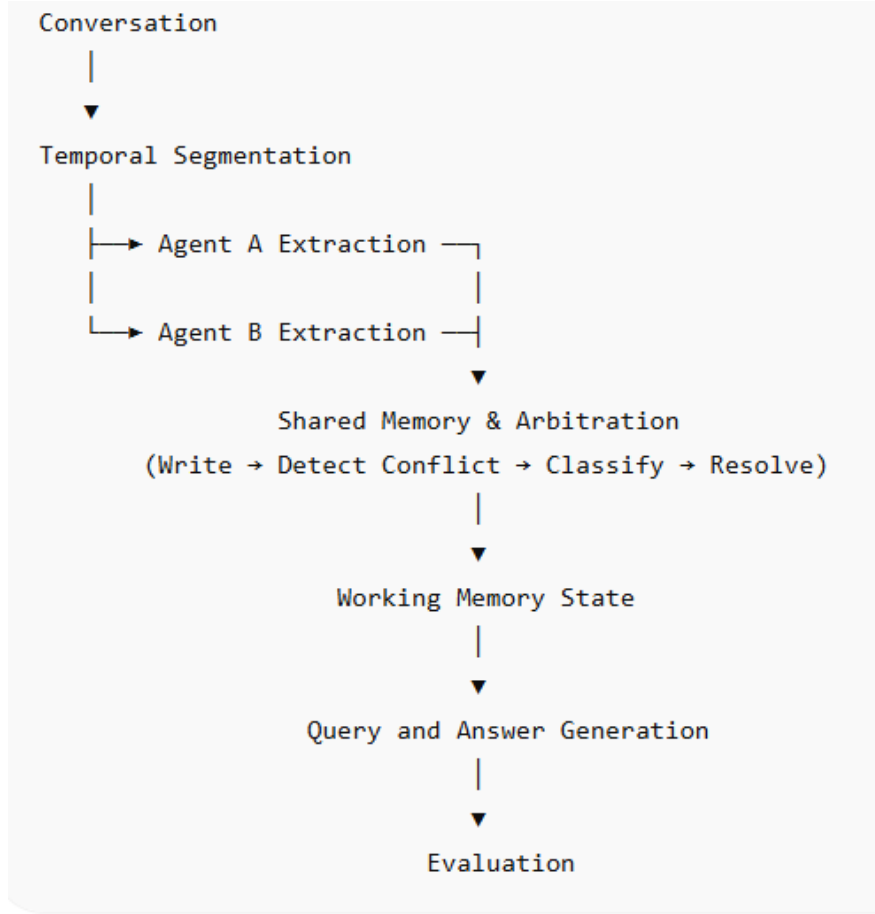
Mỗi tác tử có nhiệm vụ trích xuất các belief dưới dạng cặp khóa–giá trị kèm theo metadata như độ tin cậy, nguồn, chỉ số thời gian logic và bằng chứng gốc. Các tác tử không tự quyết định loại xung đột và cũng không tự thực hiện phân xử. Việc phát hiện, phân loại và giải quyết xung đột được chuyển về cho lớp bộ nhớ. Cách phân vai này giúp giảm sự phụ thuộc vào khả năng suy luận tự do của từng tác tử, đồng thời làm cho quá trình cập nhật bộ nhớ có thể kiểm soát và tái lập hơn.

Sau khi Agent A và Agent B hoàn tất quá trình ghi belief, Agent A truy vấn trạng thái `working_memory` đã được phân xử để sinh câu trả lời cuối cùng. Do Agent B không đọc trực tiếp bộ nhớ của Agent A trong giai đoạn trích xuất, thiết kế này cũng hạn chế hiện tượng rò rỉ thông tin giữa các tác tử trong quá trình xử lý.

4.4 Interaction Protocol

Luồng xử lý tổng thể của hệ thống bắt đầu từ một hội thoại đầu vào. Trước hết, Adapter Layer thực hiện phân đoạn hội thoại theo thời gian, tạo ra hai phần tương ứng với giai đoạn trước và sau cập nhật. Agent A xử lý phần đầu, trích xuất belief và ghi vào bộ nhớ dùng chung. Tiếp theo, Agent B xử lý phần sau và ghi các belief mới vào cùng bộ nhớ. Khi belief mới xung đột với belief đã tồn tại, `SharedMemory` kích hoạt quá trình phân loại và phân xử xung đột thông qua `ConflictClassifier` và `AdaptivePolicy`.

Sau khi quá trình phân xử hoàn tất, trạng thái `working_memory` được sử dụng làm nguồn tri thức cho bước trả lời truy vấn. Câu trả lời cuối cùng được sinh từ bộ nhớ đã được hợp nhất, thay vì từ toàn bộ hội thoại thô còn chứa các thông tin trùng lặp hoặc lỗi thời. Kết quả sau đó được đánh giá bằng các chỉ số như EM, F1, độ chính xác giải quyết xung đột và chi phí token.



Hình 2: Quy trình xử lý của hệ thống Memory Conflict Arbitration

Trong thực nghiệm, thứ tự ghi mặc định là Agent A ghi trước Agent B, tương ứng với $\theta_A < \theta_B$. Tuy nhiên, về mặt thiết kế, kết quả phân xử không được phụ thuộc vào thứ tự ghi vật lý trong bộ nhớ. Quyết định cuối cùng phải dựa trên các tín hiệu có ý nghĩa như chỉ số thời gian logic θ , độ tin cậy, nguồn thông tin và bằng chứng ngữ nghĩa. Ràng buộc này giúp hệ thống tránh phụ thuộc vào chi tiết cài đặt, đồng thời phù hợp hơn với yêu cầu của một cơ chế bộ nhớ dùng chung trong môi trường nhiều tác tử.

4.5 Temporal Split Module

Temporal Split Module được đặt trong Adapter Layer vì thao tác phân đoạn hội thoại phụ thuộc trực tiếp vào cấu trúc dữ liệu đầu vào. Module này có nhiệm vụ xác định ranh giới giữa phần hội thoại chứa thông tin cũ và phần hội thoại có khả năng chứa cập nhật mới. Khi phát hiện được tín hiệu cập nhật, hệ thống chia hội thoại tại vị trí đó. Trong trường hợp không tìm được tín hiệu rõ ràng, module sử dụng điểm giữa hội thoại làm phương án dự phòng.

Để hạn chế mất ngữ cảnh ở phần sau, đoạn hội thoại đưa vào Agent B có thể bao gồm một lượng nhỏ nội dung chồng lấn trước điểm chia. Phần chồng lấn này giúp Agent B xác định đúng thực thể đang được nhắc đến mà không cần truy cập trực tiếp vào bộ nhớ của Agent A. Nhờ đó, hệ thống vẫn duy trì được tính độc lập giữa hai tác tử trong khi giảm rủi ro trích xuất sai do thiếu ngữ cảnh.

Việc đặt Temporal Split Module trong Adapter Layer cũng giúp bảo toàn tính tổng quát của Mechanism Layer. Các tín hiệu như từ khóa cập nhật, cấu trúc lượt hội thoại hoặc chỉ số dòng đều phụ thuộc vào từng bộ dữ liệu cụ thể; do đó chúng không nên được hardcode trong cơ chế phân xử trung tâm. Mechanism Layer chỉ cần nhận các belief đã có θ , evidence và các tín hiệu cần thiết để thực hiện phân loại xung đột.

5 CƠ CHẾ PHÁT HIỆN VÀ GIẢI QUYẾT XUNG ĐỘT

5.1 ConflictClassifier

ConflictClassifier là thành phần chịu trách nhiệm xác định bản chất của xung đột khi một belief mới được ghi vào bộ nhớ. Thay vì phân loại dựa trên một belief đơn lẻ, classifier so sánh cặp belief (*existing, incoming*), kết hợp với các thông tin ngữ cảnh do Adapter Layer cung cấp, sau đó gán xung đột vào một trong bốn nhóm C1–C4.

Các tín hiệu được sử dụng gồm độ lệch thời gian logic $\Delta\theta$, nguồn sinh belief, bằng chứng văn bản, mức tương thích về phạm vi và khả năng nhập nhằng thực thể. Chẳng hạn, nếu belief mới xuất hiện muộn hơn rõ rệt và có dấu hiệu cập nhật trạng thái, xung đột có thể được xem là C1 – *temporal obsolescence*. Nếu hai nguồn gần như đồng thời đưa ra hai giá trị khác nhau, xung đột phù hợp hơn với C2 – *source disagreement*. Các trường hợp còn lại được xem xét theo khả năng lệch phạm vi hoặc nhập nhằng thực thể.

Cách thiết kế này giúp tách phần nhận diện nguyên nhân xung đột khỏi chính sách xử lý. Mechanism Layer chỉ giữ giao diện và logic phân loại ở mức khái quát, trong khi các tín hiệu phụ thuộc dữ liệu, chẳng hạn từ khóa cập nhật hay cấu trúc lượt hội thoại, được đặt ở Adapter Layer. Nhờ đó, cơ chế phân xử không bị gán cứng với một benchmark cụ thể.

5.2 Các chính sách phân xử P1–P5

Sau khi xung đột được phân loại, hệ thống sử dụng một trong các chính sách phân xử để cập nhật trạng thái bộ nhớ. Mỗi chính sách được thiết kế cho một dạng xung đột nhất định, do đó không có chính sách đơn lẻ nào được xem là tối ưu trong mọi trường hợp.

Bảng 4: Các chính sách phân xử và phạm vi áp dụng.

Mã	Chính sách	Nguyên tắc xử lý	Phạm vi phù hợp
P1	OverwritePolicy	Thay thế giá trị cũ bằng belief mới khi có bằng chứng cập nhật rõ ràng hoặc chênh lệch độ tin cậy đủ lớn.	C1, cập nhật theo thời gian.
P2	ParallelPolicy	Giữ đồng thời nhiều giá trị khi chưa đủ cơ sở để chọn một giá trị duy nhất.	C2, bất đồng nguồn.
P3	QuarantinePolicy	Tạm giữ belief mới và chưa đưa vào working memory nếu độ tin cậy hoặc bằng chứng chưa đủ mạnh.	Trường hợp thiếu chắc chắn.
P4	SoftMergePolicy	Kết hợp các giá trị khi chúng có thể được hợp nhất, đặc biệt với dữ liệu số.	Bất đồng định lượng.
P5	ContextualizePolicy	Lưu các giá trị theo phạm vi hoặc ngữ cảnh riêng thay vì loại bỏ một trong hai.	C3, lệch phạm vi.

OverwritePolicy phù hợp với các cập nhật theo thời gian, nhưng có thể làm mất thông tin đúng nếu áp dụng cho những trường hợp lệch phạm vi. ParallelPolicy an toàn hơn khi hai nguồn chưa thống nhất, nhưng không phù hợp với các fact đã lỗi thời. QuarantinePolicy đóng vai trò cơ chế phòng vệ khi bằng chứng chưa đủ rõ, trong khi SoftMergePolicy chủ yếu phù hợp với dữ liệu có thể hợp nhất. ContextualizePolicy được dùng khi hai belief đều đúng nhưng thuộc về các ngữ cảnh khác nhau.

5.3 AdaptivePolicy

AdaptivePolicy là chính sách định tuyến của hệ thống. Thay vì áp dụng cùng một cách xử lý cho mọi xung đột, thành phần này lựa chọn chính sách phù hợp dựa trên nhãn C1–C4 do ConflictClassifier cung cấp. Với C1, hệ thống ưu tiên ghi đè bằng thông tin mới; với C2, hệ thống cân nhắc giữa độ tin cậy, bằng chứng bổ trợ và khả năng giữ song song; với C3, hệ thống lưu thông tin theo ngữ cảnh; còn với C4, hệ thống cần tách khóa hoặc phân biệt lại thực thể.

Như vậy, đóng góp chính của AdaptivePolicy không nằm ở việc tạo ra một chính sách phân xử hoàn toàn mới, mà ở cơ chế lựa chọn chính sách phù hợp với từng nguyên nhân xung đột. Đây là điểm khác biệt so với các phương án single-policy, vốn thường chỉ hiệu quả với một nhóm xung đột nhất định.

6 THIẾT KẾ THỰC NGHIỆM VÀ TRIỂN KHAI

6.1 Dữ liệu đánh giá và thiết lập thực nghiệm

Thực nghiệm được tiến hành trên MemoryAgentBench CR split, cụ thể là nhánh FactConsolidation [11, 12]. Dữ liệu trong nhánh này gồm một danh sách các fact được trình bày theo thứ tự tuyến tính. Một số fact có thể được nhắc lại ở vị trí sau với giá trị khác, trong đó giá trị xuất hiện sau cùng được xem là giá trị hợp lệ. Theo mô hình xung đột của dự án, dạng dữ liệu này tương ứng chủ yếu với C1 – *temporal obsolescence*, tức thông tin cũ bị thay thế bởi một trạng thái mới hơn.

Trong phần triển khai, chỉ số thời gian logic θ được ánh xạ bằng vị trí của fact trong danh sách thay vì sử dụng thời gian thực. Do các fact trong FactConsolidation đã có cấu trúc tương đối rõ, hệ thống không sử dụng LLM để trích xuất lại fact, mà phân tích trực tiếp theo cấu trúc đầu vào. Các fact có cùng khóa nhưng khác giá trị được hợp nhất theo nguyên tắc giữ giá trị mới nhất. Sau bước hợp nhất, gpt-4o-mini được sử dụng để trả lời câu hỏi dựa trên bộ nhớ đã được làm sạch.

Baseline chính là mô hình gpt-4o-mini trả lời trực tiếp trên danh sách fact thô, trong đó vẫn còn tồn tại các fact trùng lặp hoặc lỗi thời. Cách thiết lập này giúp so sánh ảnh hưởng của cơ chế hợp nhất bộ nhớ trong điều kiện cùng backbone model, qua đó hạn chế việc trộn lẫn hiệu quả của kiến trúc với khác biệt về năng lực mô hình.

Hai mẫu được sử dụng trong lần chạy này gồm s4 thuộc nhóm single-hop và s0 thuộc nhóm multi-hop. Mỗi mẫu được đánh giá trên 100 câu hỏi. Với mỗi mẫu, dữ liệu ban đầu có khoảng 455 fact; hệ thống phát hiện và hợp nhất 143 xung đột C1, tạo ra 312 khóa sau khi loại bỏ các fact lỗi thời. Tổng cộng, thực nghiệm gồm 400 lượt trả lời trên hai phương pháp: proposed và baseline.

6.2 Baseline và chỉ số đánh giá

Thực nghiệm sử dụng hai nhóm so sánh. Nhóm thứ nhất là baseline end-to-end với cùng mô hình gpt-4o-mini. Đây là baseline chính để đánh giá tác động của kiến trúc proposed đối với độ chính xác và chi phí token. Nhóm thứ hai là các single-policy baseline như OverwritePolicy, ParallelPolicy, QuarantinePolicy, SoftMergePolicy và ContextualizePolicy. Các baseline này được dùng cho thiết kế ablation nhằm kiểm tra giả thuyết rằng không một chính sách đơn lẻ nào phù hợp với mọi loại xung đột.

Các chỉ số chính gồm CR-Acc, Exact Match (EM), số token trung bình trên mỗi câu hỏi và tỷ lệ chi phí so với baseline cùng mô hình. Trong thực nghiệm hiện tại, CR-Acc và EM có cùng giá trị vì câu trả lời được chấm theo khớp chính xác với đáp án chuẩn. Bên cạnh đó, token cost được dùng để đánh giá liệu việc hợp nhất bộ nhớ trước khi truy vấn có giúp giảm độ dài ngữ cảnh đưa vào LLM hay không.

6.3 Kết quả thực nghiệm

Bảng 5 trình bày kết quả trên hai mẫu s4 và s0. Ở cả hai trường hợp, phương pháp proposed đạt độ chính xác cao hơn baseline, đồng thời sử dụng ít token hơn.

Bảng 5: Kết quả trên MemoryAgentBench CR split với gpt-4o-mini.

Sample	Hop	Method	CR-Acc	EM	Avg tokens
s4	single-hop	Proposed	0.96	0.96	3,666
s4	single-hop	Baseline	0.49	0.49	6,064
s0	multi-hop	Proposed	0.23	0.23	3,675
s0	multi-hop	Baseline	0.09	0.09	6,074

Trên mẫu s4 single-hop, proposed đạt CR-Acc/EM bằng 0.96, trong khi baseline chỉ đạt 0.49. Kết quả này cho thấy việc loại bỏ các fact lỗi thời trước khi truy vấn giúp mô hình tránh bị nhiễu bởi các thông tin mâu thuẫn trong ngữ cảnh đầu vào. Với mẫu s0 multi-hop, proposed đạt 0.23 so với 0.09 của baseline. Mặc dù mức điểm tuyệt đối còn thấp, kết quả vẫn cho thấy cơ chế hợp nhất bộ nhớ đem lại lợi ích trong bối cảnh cần suy luận qua nhiều fact.

Bảng 6: Mức cải thiện độ chính xác và chi phí token.

Sample	Hop	Acc gain	Relative gain	Token saving
s4	single-hop	+0.47	1.96×	39.5%
s0	multi-hop	+0.14	2.56×	39.5%

Về chi phí, proposed sử dụng khoảng 3.67k token cho mỗi câu hỏi, trong khi baseline sử dụng khoảng 6.07k token. Tỷ lệ chi phí xấp xỉ 0.60 ở cả single-hop và multi-hop, tương đương mức tiết kiệm khoảng 39.5%. Điều này phản ánh lợi ích trực tiếp của việc đưa vào LLM một trạng thái bộ nhớ đã được hợp nhất thay vì toàn bộ danh sách fact thô.

Tuy nhiên, kết quả cần được diễn giải trong phạm vi của CR split. Do benchmark này chủ yếu kiểm tra xung đột C1, thực nghiệm hiện tại xác nhận rõ hiệu quả của cơ chế loại bỏ thông tin lỗi thời, nhưng chưa đủ để kết luận đầy đủ về toàn bộ AdaptivePolicy trên bốn nhóm C1–C4. Để kiểm chứng đầy đủ giả thuyết định tuyến chính sách, cần bổ sung các mẫu có C2, C3 và C4 hoặc xây dựng thêm tập injected subset có kiểm soát.

6.4 Đối chiếu câu hỏi nghiên cứu với kết quả đạt được

Dựa trên thiết kế hệ thống và kết quả thực nghiệm hiện tại, bốn câu hỏi nghiên cứu được trả lời ở các mức độ khác nhau. Bảng 7 tóm tắt phần đã được giải quyết, bằng chứng tương ứng và phần còn hạn chế.

Bảng 7: Mức độ giải quyết các câu hỏi nghiên cứu trong phạm vi dự án.

RQ	Vấn đề đặt ra	Mức độ giải quyết	Bằng chứng và giới hạn
RQ1	Phát hiện conflict dựa trên cặp belief thay vì từng belief đơn lẻ.	Đã giải quyết ở mức thiết kế và triển khai cơ chế.	SharedMemory.write chỉ kích hoạt phân loại khi cùng key nhưng khác value; classifier nhận cặp (<i>existing, incoming</i>), nên conflict type không do agent tự gán.
RQ2	Taxonomy C1–C4 có đủ biểu diễn các root causes chính của conflict trong hệ hai agent hay không.	Giải quyết một phần.	Báo cáo đã định nghĩa C1–C4 và giải thích policy tương ứng. Tuy nhiên, thực nghiệm hiện tại chủ yếu kiểm tra C1; C2–C4 cần thêm dữ liệu đánh giá.
RQ3	AdaptivePolicy có vượt single-policy baseline khi phân tích theo từng conflict type hay không.	Chưa chứng minh đầy đủ.	Cơ chế định tuyến đã được thiết kế, nhưng kết quả hiện tại chưa có breakdown đủ cho C2, C3 và C4. Vì vậy, chưa thể kết luận mạnh về ưu thế toàn phần của AdaptivePolicy.
RQ4	Kiến trúc extract–arbitrate–query có thể giảm token cost so với baseline end-to-end cùng backbone model hay không.	Đã có bằng chứng ban đầu.	Trên hai mẫu s4 và s0, proposed giảm token trung bình từ khoảng 6.07k xuống 3.67k, tương đương tiết kiệm xấp xỉ 39.5%.

Từ bảng trên, có thể kết luận rằng dự án đã giải quyết rõ nhất RQ1 và RQ4. RQ2 đã được giải quyết ở mức khung lý thuyết và thiết kế cơ chế, nhưng cần dữ liệu đa dạng hơn để kiểm chứng đầy đủ. RQ3 là câu hỏi còn mở nhất trong phạm vi hiện tại, vì kết quả thực nghiệm chưa đủ bao phủ tất cả conflict type. Do đó, claim hợp lý của báo cáo nên được viết theo hướng: hệ thống đã chứng minh hiệu quả ban đầu với xung đột C1 và có kiến trúc sẵn sàng mở rộng cho C2–C4, thay vì khẳng định AdaptivePolicy đã vượt mọi baseline trong mọi loại xung đột.

7 KẾT LUẬN

Báo cáo đã trình bày hệ thống *Memory Conflict Arbitration System*, một cơ chế bộ nhớ dùng chung cho hai tác tử độc lập nhằm phát hiện, phân loại và xử lý các xung đột phát sinh trong quá trình cập nhật belief. Thay vì để các tác tử tự quyết định cách giải quyết mâu thuẫn hoặc để bộ nhớ ghi đè theo thứ tự ghi vật lý, hệ thống tách riêng quá trình phân xử thành một lớp xử lý có cấu trúc, bao gồm schema biểu diễn belief, taxonomy xung đột, các chính sách phân xử và cơ chế lưu vết lịch sử.

Đóng góp chính của dự án nằm ở cách tổ chức quá trình phân xử theo nguyên nhân xung đột. Các chính sách đơn lẻ thường chỉ phù hợp với một nhóm tình huống nhất định: ghi đè phù hợp với thông tin lỗi thời theo thời gian, giữ song song phù hợp với bất đồng nguồn, còn lưu theo ngữ cảnh phù hợp với các trường hợp lệch phạm vi. Do đó, hệ thống sử dụng *AdaptivePolicy* để lựa chọn chính sách xử lý dựa trên nhãn xung đột C1–C4, thay vì áp dụng một quy tắc cố định cho mọi trường hợp.

Về mặt kiến trúc, dự án phân tách giữa *Mechanism Layer* và *Adapter Layer*. *Mechanism Layer* chứa các thành phần xử lý cốt lõi và độc lập với dữ liệu đánh giá, trong khi *Adapter Layer* chịu trách nhiệm ánh xạ từng benchmark cụ thể thành các tín hiệu đầu vào cho cơ chế phân xử. Cách tổ chức này giúp hạn chế sự phụ thuộc vào một bộ dữ liệu riêng lẻ và tạo điều kiện để mở rộng hệ thống sang các bối cảnh đánh giá khác.

Thực nghiệm trên *MemoryAgentBench* CR split với gpt-4o-mini cho thấy cơ chế hợp nhất bộ nhớ đem lại cải thiện rõ rệt trong nhóm xung đột C1. Ở thiết lập single-hop, hệ thống đạt CR-Acc/EM bằng 0.96 so với 0.49 của baseline end-to-end. Ở thiết lập multi-hop, hệ thống đạt 0.23 so với 0.09 của baseline. Đồng thời, số token trung bình giảm từ khoảng 6.07k xuống còn khoảng 3.67k token cho mỗi câu hỏi, tương đương mức tiết kiệm xấp xỉ 40%. Kết quả này cho thấy việc loại bỏ các fact lỗi thời trước khi truy vấn có thể giúp mô hình giảm nhiễu ngữ cảnh và sử dụng chi phí hiệu quả hơn.

Tuy nhiên, phạm vi kết luận của thực nghiệm hiện tại cần được giới hạn rõ ràng. CR split chủ yếu kiểm tra dạng xung đột C1 – *temporal obsolescence*, nên kết quả hiện tại mới xác nhận hiệu quả của cơ chế xử lý thông tin lỗi thời, chưa đủ để chứng minh đầy đủ ưu thế của *AdaptivePolicy* trên toàn bộ bốn nhóm C1–C4. Để đánh giá hoàn chỉnh hơn, cần bổ sung các mẫu có C2 – *source disagreement*, C3 – *scope mismatch* và C4 – *entity ambiguity*, hoặc xây dựng thêm tập dữ liệu có kiểm soát cho từng loại xung đột.

Nhìn chung, trong phạm vi một dự án học phần, hệ thống đã hình thành được một khung xử lý tương đối rõ ràng cho bài toán điều phối xung đột bộ nhớ trong hệ đa tác tử. Kết quả ban đầu cho thấy hướng tiếp cận dựa trên phân xử bộ nhớ có tiềm năng cải thiện độ chính xác và giảm chi phí token trong các bài toán fact consolidation. Các bước phát triển tiếp theo nên tập trung vào mở rộng tập đánh giá, kiểm tra đầy đủ bốn nhóm xung đột và phân tích sâu hơn các lỗi phát sinh trong truy vấn multi-hop.

TÀI LIỆU THAM KHẢO

- [1] Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*.
- [2] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.
- [3] Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. arXiv preprint arXiv:2303.11366.
- [4] Li, G., Hammoud, H. A. K., Itani, H., Khizbullin, D., & Ghanem, B. (2023). *CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society*. arXiv preprint arXiv:2303.17760.
- [5] Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., & Wang, C. (2023). *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. arXiv preprint arXiv:2308.08155.
- [6] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*.
- [7] Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). *MemGPT: Towards LLMs as Operating Systems*. arXiv preprint arXiv:2310.08560.
- [8] Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7), 558–565.
- [9] Alchourrón, C. E., Gärdenfors, P., & Makinson, D. (1985). On the logic of theory change: Partial meet contraction and revision functions. *The Journal of Symbolic Logic*, 50(2), 510–530.
- [10] Doyle, J. (1979). A truth maintenance system. *Artificial Intelligence*, 12(3), 231–272.
- [11] Hu, Y., Wang, Y., & McAuley, J. (2025). *Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions*. arXiv preprint arXiv:2507.05257.
- [12] HUST-AI-HYZ. (2025). *MemoryAgentBench: Dataset and code repository*. Hugging Face Datasets and GitHub repository.